

The Ultimate Sampling Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	3
2	Part 0 — The 30-Second Exam Strategy	4
3	Part 1 — Foundations	4
3.1	1.1 Why sampling?	4
3.2	1.2 Key vocabulary	4
3.3	1.3 Common parameters and their estimators	5
3.4	1.4 Bias, variance, MSE	5
3.5	1.5 Finite Population Correction (FPC)	5
3.6	1.6 Variance of the sample mean (SRS)	5
3.7	1.7 Confidence interval for μ	5
3.8	1.8 Independence of the sample	6
3.9	1.9 Sampling designs at a glance	6
4	Part 2 — Building a Finite Population	6
4.1	2.1 A simple finite population	6
4.2	2.2 A stratified population	6
4.3	2.3 Population as a discrete distribution (exam style)	7
4.4	2.4 Summary table by stratum	7
5	Part 3 — Simple Random Sampling (SRS)	8
5.1	3.1 Draw an SRS sample	8
5.2	3.2 Variance with FPC	8
5.3	3.3 Confidence interval for μ	9
5.4	3.4 Estimate the population total	9
5.5	3.5 SRS for a proportion	9
5.6	3.6 Sample size for a target margin of error	10
5.7	3.7 SRS summary	10
6	Part 4 — Stratified Sampling	10
6.1	4.1 What it is	10
6.2	4.2 Proportional allocation	11
6.3	4.3 Draw the stratified sample and estimate	11
6.4	4.4 Confidence interval	12
6.5	4.5 Optimal (Neyman) allocation	12
6.6	4.6 Optimal with costs	13
6.7	4.7 Optimal allocation vs proportional: total sample size for target variance	13
6.8	4.8 Stratified sampling summary	13
7	Part 5 — Systematic Sampling	13

7.1	5.1 What it is	13
7.2	5.2 Draw a systematic sample	14
7.3	5.3 Variance under systematic sampling	14
7.4	5.4 When systematic beats SRS	14
7.5	5.5 Systematic sampling summary	15
8	Part 6 — Cluster Sampling	15
8.1	6.1 What it is	15
8.2	6.2 Cluster sampling vs SRS	15
8.3	6.3 Build a clustered population	15
8.4	6.4 One-stage cluster sampling	15
8.5	6.5 Variance under cluster sampling	16
8.6	6.6 Design effect for cluster sampling	17
8.7	6.7 Cluster sampling summary	17
9	Part 7 — Two-Stage Sampling	18
9.1	7.1 What it is	18
9.2	7.2 Two-stage sample	18
9.3	7.3 Variance	18
9.4	7.4 Two-stage summary	19
10	Part 8 — Ratio and Regression Estimators	19
10.1	8.1 Ratio estimator	19
10.2	8.2 Regression estimator	20
10.3	8.3 Reporting	21
11	Part 9 — Bias, MSE, and Relative Efficiency	21
11.1	9.1 Simulate bias, variance, MSE	21
11.2	9.2 Relative efficiency	22
11.3	9.3 Bias of the ratio estimator (empirical)	22
11.4	9.4 Comparing estimators summary	23
12	Part 10 — Coverage Probability Simulations	23
12.1	10.1 Coverage of a 95% CI for μ (SRS)	23
12.2	10.2 Coverage with FPC vs without	23
12.3	10.3 Coverage under misspecified distribution	24
13	Part 11 — Sample Size Determination	25
13.1	11.1 For a target variance of the mean	25
13.2	11.2 For a target margin of error (with FPC)	25
13.3	11.3 For a proportion	26
13.4	11.4 Stratified sample size for target variance (proportional)	26
13.5	11.5 Sample size for Neyman allocation	26
13.6	11.6 Sample size summary	27
14	Part 12 — Design Effects	27
14.1	12.1 Definition	27
14.2	12.2 Effective sample size	27
14.3	12.3 deff for stratified sampling	27
14.4	12.4 deff for cluster sampling	28
14.5	12.5 Reporting	28
15	Part 13 — Horvitz-Thompson Estimator	28
15.1	13.1 Definition	28
15.2	13.2 Variance of HT estimator	28

15.3	13.3 For SRS	29
15.4	13.4 Unequal probability sampling	29
15.5	13.5 HT summary	29
16	Part 14 — Probability Proportional to Size (PPS)	30
16.1	14.1 What it is	30
16.2	14.2 PPS with replacement	30
16.3	14.3 Variance of HH estimator	30
16.4	14.4 PPS summary	31
17	Part 15 — Non-Response	31
17.1	15.1 Types of non-response	31
17.2	15.2 Effects on estimates	31
17.3	15.3 Weighting adjustment	32
17.4	15.4 Summary	32
18	Part 16 — Bootstrap Variance Estimation	32
18.1	16.1 Why bootstrap?	32
18.2	16.2 SRS bootstrap	33
18.3	16.3 Bootstrap CI	33
18.4	16.4 Cluster bootstrap	34
18.5	16.5 Bootstrap summary	34
19	Part 17 — Theory Q&A	34
20	Part 18 — Exam Scenarios	36
20.1	18.1 “Estimate the population mean under SRS”	36
20.2	18.2 “Compare SRS and stratified variance”	36
20.3	18.3 “Optimal allocation”	36
20.4	18.4 “Bias and MSE of an estimator”	36
20.5	18.5 “Coverage of a CI”	36
20.6	18.6 “Sample size for target precision”	36
20.7	18.7 “Design effect”	37
20.8	18.8 “Cluster sampling”	37
20.9	18.9 “Ratio/regression estimator”	37
20.10	18.10 “Non-response adjustment”	37
21	Part 19 — Quick Reference	37
22	Part 20 — Common Mistakes	38
23	Part 21 — The Single Most Important Idea	39
23.1	What is covered	39

1 How to Use This Cheat Sheet

Topic	Section
30-second strategy	0
Foundations of sampling theory	1
Building a finite population	2
Simple Random Sampling (SRS)	3
Stratified sampling	4
Systematic sampling	5
Cluster sampling	6

Topic	Section
Two-stage sampling	7
Ratio and regression estimators	8
Bias, MSE, and relative efficiency	9
Coverage probability simulations	10
Sample size determination	11
Design effects	12
Horvitz-Thompson estimator	13
Probability proportional to size (PPS)	14
Non-response	15
Bootstrap variance	16
Theory Q&A	17
Exam scenarios	18
Quick reference	19
Common mistakes	20

Golden rule: every chunk is self-contained.

2 Part 0 — The 30-Second Exam Strategy

For **any** sampling question, run this pipeline:

1. **Identify the design:** SRS, stratified, systematic, cluster, two-stage, PPS?
2. **Identify the parameter:** population mean, total, proportion?
3. **Compute the estimator** (usually a weighted mean or proportion).
4. **Compute the variance** using the design-specific formula, including FPC.
5. **Build a CI** with the normal approximation or t.
6. **Simulate** to check bias, variance, MSE, coverage.
7. **Compare designs** using relative efficiency or design effect.

If you can do these seven steps, you can answer any sampling question.

3 Part 1 — Foundations

3.1 1.1 Why sampling?

A **population** has N units. Measuring all of them (census) is expensive. A **sample** of n units estimates the population parameters.

Goal: Estimate a population **parameter** ($\mu, \tau = N\mu, P$) from a **sample statistic** ($\bar{y}, \hat{\tau}, \hat{p}$).

3.2 1.2 Key vocabulary

Term	Meaning
Population	Complete set of N units
Sampling frame	List of all units in the population
Sample	Subset of n units selected from the frame
Parameter	Population value (fixed, unknown): μ, τ, P
Statistic	Sample value (random): $\bar{y}, \hat{\tau}, \hat{p}$

Term	Meaning
Sampling design	Rule for selecting the sample
Inclusion probability π_i	Probability unit i is in the sample
Design weight $w_i = 1/\pi_i$	How many population units i represents

3.3 1.3 Common parameters and their estimators

Parameter	Symbol	Estimator
Population mean	$\mu = \frac{1}{N} \sum_{i=1}^N y_i$	$\bar{y} = \frac{1}{n} \sum_{i \in S} y_i$
Population total	$\tau = \sum_{i=1}^N y_i = N\mu$	$\hat{\tau} = N\bar{y}$
Population proportion	$P = \frac{1}{N} \sum_{i=1}^N y_i$ (binary)	$\hat{p} = \bar{y}$ (binary)
Population variance	$S^2 = \frac{1}{N-1} \sum_{i=1}^N (y_i - \mu)^2$	$s^2 = \frac{1}{n-1} \sum_{i \in S} (y_i - \bar{y})^2$

3.4 1.4 Bias, variance, MSE

- **Bias:** $\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta$.
- **Variance:** $\text{Var}(\hat{\theta}) = E[(\hat{\theta} - E[\hat{\theta}])^2]$.
- **Mean Squared Error:** $\text{MSE}(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = \text{Bias}^2 + \text{Variance}$.

Unbiased = Bias = 0. **Consistent** = bias and variance $\rightarrow 0$ as $n \rightarrow \infty$. **Relative efficiency of $\hat{\theta}_1$ vs $\hat{\theta}_2$:**
 $\text{RE} = \text{MSE}(\hat{\theta}_2)/\text{MSE}(\hat{\theta}_1)$.

3.5 1.5 Finite Population Correction (FPC)

$$f = \frac{n}{N} \quad (\text{sampling fraction})$$

$$\text{FPC} = 1 - \frac{n}{N}$$

Effect: Reduces the variance of the estimator because when you've sampled most of the population, there's less uncertainty about the mean.

- $n \ll N$: FPC $\rightarrow 1$ ignore it.
- $n = N$: FPC = 0 $\rightarrow$ zero variance (census, no uncertainty).

3.6 1.6 Variance of the sample mean (SRS)

$$\text{Var}(\bar{y}) = \left(1 - \frac{n}{N}\right) \frac{S^2}{n}$$

where S^2 is the **population** variance. In practice we don't know S^2 , so we estimate:

$$\widehat{\text{Var}}(\bar{y}) = \left(1 - \frac{n}{N}\right) \frac{s^2}{n}$$

3.7 1.7 Confidence interval for μ

$$\bar{y} \pm z_{\alpha/2} \sqrt{\left(1 - \frac{n}{N}\right) \frac{s^2}{n}}$$

For small n , use t_{n-1} instead of z .

3.8 1.8 Independence of the sample

Independence means one unit's selection doesn't affect another's. In SRS without replacement, units are **not independent** (but **exchangeable**). Design-based inference handles this using **inclusion probabilities** instead of independence.

3.9 1.9 Sampling designs at a glance

Design	When to use	Key feature
SRS	Simple, no auxiliary info	Equal $\pi_i = n/N$
Stratified	Population has distinct subgroups	Sample within strata
Systematic	Ordered frame, quick	Every k -th unit
Cluster	Units geographically grouped	Sample clusters, not units
Two-stage	Clusters + units	Sample PSUs then SSUs
PPS	Sizes vary widely	Larger units more likely

4 Part 2 — Building a Finite Population

4.1 2.1 A simple finite population

```
# --- Self-contained: build finite population ---
set.seed(201)
p2_N    <- 1000
p2_pop  <- rnorm(p2_N, mean = 50, sd = 10)

p2_mu   <- mean(p2_pop)
p2_S2   <- var(p2_pop)
p2_sigma <- sqrt(p2_S2)

round(c(mu = p2_mu, var = p2_S2, sd = p2_sigma), 4)

##      mu      var      sd
## 50.6748 105.5262 10.2726
```

4.2 2.2 A stratified population

```
# --- Self-contained: stratified population ---
set.seed(202)
p2_N1 <- 100; p2_N2 <- 200; p2_N3 <- 300
p2_N  <- p2_N1 + p2_N2 + p2_N3

p2_s1 <- rnorm(p2_N1, 10, 2)
p2_s2 <- rnorm(p2_N2, 12, 3)
p2_s3 <- rnorm(p2_N3, 15, 4)

p2_pop  <- c(p2_s1, p2_s2, p2_s3)
p2_strata <- c(rep(1, p2_N1), rep(2, p2_N2), rep(3, p2_N3))

# Population parameters
p2_mu <- mean(p2_pop)
```

```

p2_S2      <- var(p2_pop)
p2_strata_N <- c(p2_N1, p2_N2, p2_N3)
p2_strata_W <- p2_strata_N / p2_N

# Stratum-specific means and variances
p2_strata_stats <- tibble(
  stratum = 1:3,
  N_h     = p2_strata_N,
  W_h     = p2_strata_W,
  mu_h    = c(mean(p2_s1), mean(p2_s2), mean(p2_s3)),
  S2_h    = c(var(p2_s1), var(p2_s2), var(p2_s3))
)
kable(p2_strata_stats, digits = 4)

```

stratum	N_h	W_h	mu_h	S2_h
1	100	0.1667	10.1776	3.9566
2	200	0.3333	12.1158	9.8791
3	300	0.5000	14.5704	16.3335

4.3 2.3 Population as a discrete distribution (exam style)

```

# --- Self-contained: discrete population with strata ---
set.seed(203)
p2_N1 <- 100; p2_N2 <- 200; p2_N3 <- 300
p2_N  <- p2_N1 + p2_N2 + p2_N3

p2_s1 <- sample(1:5, p2_N1, replace = TRUE, prob = rep(0.2, 5))
p2_s2 <- sample(1:5, p2_N2, replace = TRUE, prob = c(0.1, 0.1, 0.1, 0.1, 0.6))
p2_s3 <- sample(1:5, p2_N3, replace = TRUE, prob = c(0.1, 0.3, 0.3, 0.1, 0.2))

p2_pop <- c(p2_s1, p2_s2, p2_s3)
p2_strata <- c(rep(1, p2_N1), rep(2, p2_N2), rep(3, p2_N3))
p2_mu <- mean(p2_pop)

round(p2_mu, 4)

```

```
## [1] 3.3267
```

```
table(p2_strata, p2_pop) %>% kable()
```

	1	2	3	4	5
24	20	19	23	14	
27	26	16	16	115	
26	85	77	40	72	

4.4 2.4 Summary table by stratum

```

# --- Self-contained: summary by stratum ---
set.seed(204)
p2_N1 <- 100; p2_N2 <- 200; p2_N3 <- 300
p2_s1 <- rnorm(p2_N1, 10, 2)

```

```

p2_s2 <- rnorm(p2_N2, 12, 3)
p2_s3 <- rnorm(p2_N3, 15, 4)
p2_pop <- c(p2_s1, p2_s2, p2_s3)
p2_strata <- c(rep(1, p2_N1), rep(2, p2_N2), rep(3, p2_N3))

tibble(y = p2_pop, h = factor(p2_strata)) %>%
  group_by(h) %>%
  summarise(n = n(), mean = mean(y), sd = sd(y), .groups = "drop") %>%
  kable(digits = 3)

```

h	n	mean	sd
1	100	9.784	1.884
2	200	12.221	3.081
3	300	15.192	3.508

5 Part 3 — Simple Random Sampling (SRS)

5.1 3.1 Draw an SRS sample

```

# --- Self-contained: SRS sample ---
set.seed(301)
p3_N <- 500
p3_pop <- rnorm(p3_N, 50, 10)

p3_n <- 50
p3_srs <- sample(p3_pop, p3_n, replace = FALSE)

p3_ybar <- mean(p3_srs)
p3_s2 <- var(p3_srs)

round(c(ybar = p3_ybar, s2 = p3_s2), 4)

##      ybar      s2
## 52.2279 63.7125

```

5.2 3.2 Variance with FPC

```

# --- Self-contained: variance of ybar ---
set.seed(302)
p3_N <- 500
p3_pop <- rnorm(p3_N, 50, 10)
p3_n <- 50
p3_srs <- sample(p3_pop, p3_n)
p3_ybar <- mean(p3_srs)
p3_s2 <- var(p3_srs)

p3_Var_ybar <- (1 - p3_n / p3_N) * p3_s2 / p3_n
p3_SE_ybar <- sqrt(p3_Var_ybar)

round(c(ybar = p3_ybar, var_ybar = p3_Var_ybar, se = p3_SE_ybar), 4)

```

```
##      ybar var_ybar      se
## 48.9064  1.9599  1.4000
```

5.3 3.3 Confidence interval for μ

```
# --- Self-contained: 95% CI for mu ---
set.seed(303)
p3_N   <- 500
p3_pop <- rnorm(p3_N, 50, 10)
p3_n   <- 50
p3_srs <- sample(p3_pop, p3_n)
p3_ybar <- mean(p3_srs)
p3_s2   <- var(p3_srs)
p3_se   <- sqrt((1 - p3_n / p3_N) * p3_s2 / p3_n)
p3_z    <- qnorm(0.975)

p3_ci <- c(p3_ybar - p3_z * p3_se, p3_ybar + p3_z * p3_se)
round(p3_ci, 4)
```

```
## [1] 47.8793 53.2674
```

Report template: “The 95% CI for the population mean is [lo,hi].”

5.4 3.4 Estimate the population total

```
# --- Self-contained: total estimate ---
set.seed(304)
p3_N   <- 500
p3_pop <- rnorm(p3_N, 50, 10)
p3_n   <- 50
p3_srs <- sample(p3_pop, p3_n)

p3_tau_hat <- p3_N * mean(p3_srs)
p3_Var_tau <- p3_N^2 * (1 - p3_n / p3_N) * var(p3_srs) / p3_n

round(c(total = p3_tau_hat, se_total = sqrt(p3_Var_tau)), 4)
```

```
##      total  se_total
## 24987.8134  680.1808
```

5.5 3.5 SRS for a proportion

```
# --- Self-contained: proportion ---
set.seed(305)
p3_N   <- 1000
p3_pop <- rbinom(p3_N, 1, 0.4) # 40% successes
p3_n   <- 100
p3_srs <- sample(p3_pop, p3_n)

p3_phat <- mean(p3_srs)
p3_Var_ph <- (1 - p3_n / p3_N) * p3_phat * (1 - p3_phat) / (p3_n - 1)
p3_SE_ph <- sqrt(p3_Var_ph)

round(c(phat = p3_phat, se = p3_SE_ph), 4)
```

```
## phat    se
## 0.370 0.046
```

5.6 3.6 Sample size for a target margin of error

$$n_0 = \frac{z^2 S^2}{e^2}, \quad n = \frac{n_0}{1 + n_0/N}$$

```
# --- Self-contained: sample size for SRS ---
p3_N <- 500
p3_S2 <- 100 # assume known or guessed from pilot
p3_e <- 2 # target margin of error
p3_z <- qnorm(0.975)

p3_n0 <- p3_z^2 * p3_S2 / p3_e^2
p3_n <- ceiling(p3_n0 / (1 + p3_n0 / p3_N))
p3_n
```

```
## [1] 81
```

Interpretation: To achieve a margin of error of 2 units, sample at least n units.

5.7 3.7 SRS summary

Property	Value
Inclusion probability π_i	n/N for all i
Estimator of mean	$\bar{y}$
Estimator of total	$N\bar{y}$
Var of mean	$(1 - n/N)S^2/n$
Var of total	$N^2(1 - n/N)S^2/n$
Best for	No prior information, symmetric units

6 Part 4 — Stratified Sampling

6.1 4.1 What it is

- Divide the population into H non-overlapping **strata**.
- Sample independently from each stratum.
- Combine stratum estimates using population weights.

Estimator:

$$\bar{y}_{st} = \sum_{h=1}^H W_h \bar{y}_h, \quad W_h = N_h/N$$

Variance:

$$\text{Var}(\bar{y}_{st}) = \sum_{h=1}^H W_h^2 \left(1 - \frac{n_h}{N_h}\right) \frac{S_h^2}{n_h}$$

6.2 4.2 Proportional allocation

$$n_h = n \cdot \frac{N_h}{N}$$

Sample size per stratum is proportional to stratum size.

```
# --- Self-contained: proportional allocation ---
set.seed(401)
p4_N1 <- 100; p4_N2 <- 200; p4_N3 <- 300
p4_N  <- p4_N1 + p4_N2 + p4_N3
p4_s1 <- rnorm(p4_N1, 10, 2)
p4_s2 <- rnorm(p4_N2, 12, 3)
p4_s3 <- rnorm(p4_N3, 15, 4)

p4_n    <- 60
p4_W    <- c(p4_N1, p4_N2, p4_N3) / p4_N
p4_n_h  <- round(p4_n * p4_W)
p4_n_h
```

```
## [1] 10 20 30
```

```
sum(p4_n_h)
```

```
## [1] 60
```

6.3 4.3 Draw the stratified sample and estimate

```
# --- Self-contained: stratified estimate ---
set.seed(402)
p4_N1 <- 100; p4_N2 <- 200; p4_N3 <- 300
p4_N  <- p4_N1 + p4_N2 + p4_N3
p4_s1 <- rnorm(p4_N1, 10, 2)
p4_s2 <- rnorm(p4_N2, 12, 3)
p4_s3 <- rnorm(p4_N3, 15, 4)

p4_n    <- 60
p4_W    <- c(p4_N1, p4_N2, p4_N3) / p4_N
p4_n_h  <- round(p4_n * p4_W)

p4_samp1 <- sample(p4_s1, p4_n_h[1])
p4_samp2 <- sample(p4_s2, p4_n_h[2])
p4_samp3 <- sample(p4_s3, p4_n_h[3])

p4_ybar_h <- c(mean(p4_samp1), mean(p4_samp2), mean(p4_samp3))
p4_s2_h   <- c(var(p4_samp1), var(p4_samp2), var(p4_samp3))

p4_ybar_st <- sum(p4_W * p4_ybar_h)
p4_Var_st  <- sum(p4_W^2 * (1 - p4_n_h / c(p4_N1, p4_N2, p4_N3)) *
                  p4_s2_h / p4_n_h)
p4_SE_st   <- sqrt(p4_Var_st)

round(c(ybar_st = p4_ybar_st, var_st = p4_Var_st, se_st = p4_SE_st), 4)
```

```
## ybar_st var_st se_st
## 13.5523 0.1771 0.4208
```

6.4 4.4 Confidence interval

```
# --- Self-contained: stratified CI ---
set.seed(403)
p4_N1 <- 100; p4_N2 <- 200; p4_N3 <- 300
p4_N <- p4_N1 + p4_N2 + p4_N3
p4_s1 <- rnorm(p4_N1, 10, 2); p4_s2 <- rnorm(p4_N2, 12, 3); p4_s3 <- rnorm(p4_N3, 15, 4)
p4_n <- 60
p4_W <- c(p4_N1, p4_N2, p4_N3) / p4_N
p4_n_h <- round(p4_n * p4_W)

p4_samp1 <- sample(p4_s1, p4_n_h[1])
p4_samp2 <- sample(p4_s2, p4_n_h[2])
p4_samp3 <- sample(p4_s3, p4_n_h[3])
p4_ybar_h <- c(mean(p4_samp1), mean(p4_samp2), mean(p4_samp3))
p4_s2_h <- c(var(p4_samp1), var(p4_samp2), var(p4_samp3))
p4_ybar_st <- sum(p4_W * p4_ybar_h)
p4_SE_st <- sqrt(sum(p4_W^2 * (1 - p4_n_h / c(p4_N1, p4_N2, p4_N3)) *
                    p4_s2_h / p4_n_h))

p4_z <- qnorm(0.975)
round(c(p4_ybar_st - p4_z * p4_SE_st,
        p4_ybar_st + p4_z * p4_SE_st), 4)

## [1] 12.9764 14.5532
```

6.5 4.5 Optimal (Neyman) allocation

Minimizes variance for fixed n :

$$n_h = n \cdot \frac{N_h S_h}{\sum_j N_j S_j}$$

With different costs per stratum:

$$n_h = n \cdot \frac{N_h S_h / \sqrt{c_h}}{\sum_j N_j S_j / \sqrt{c_j}}$$

```
# --- Self-contained: Neyman allocation ---
set.seed(404)
p4_N1 <- 100; p4_N2 <- 200; p4_N3 <- 300
p4_S1 <- 2; p4_S2 <- 3; p4_S3 <- 4 # population SDs
p4_N_h <- c(p4_N1, p4_N2, p4_N3)
p4_S_h <- c(p4_S1, p4_S2, p4_S3)
p4_n <- 60

p4_alpha_h <- p4_N_h * p4_S_h / sum(p4_N_h * p4_S_h)
p4_n_h_opt <- round(p4_n * p4_alpha_h)
p4_n_h_opt

## [1] 6 18 36

sum(p4_n_h_opt)

## [1] 60
```

6.6 4.6 Optimal with costs

```
# --- Self-contained: Neyman with costs ---
p4_N_h <- c(100, 200, 300)
p4_S_h <- c(2, 3, 4)
p4_c_h <- c(1, 2, 4)      # cost per unit in each stratum
p4_n    <- 60

p4_alpha <- (p4_N_h * p4_S_h / sqrt(p4_c_h)) /
             sum(p4_N_h * p4_S_h / sqrt(p4_c_h))
p4_n_h_opt_cost <- round(p4_n * p4_alpha)
p4_n_h_opt_cost

## [1] 10 21 29
sum(p4_n_h_opt_cost)

## [1] 60
```

6.7 4.7 Optimal allocation vs proportional: total sample size for target variance

For a target variance V :

$$n = \frac{(\sum_h W_h S_h)^2}{V + \frac{1}{N} \sum_h W_h S_h^2} \quad (\text{proportional})$$

$$n = \frac{(\sum_h W_h S_h)^2}{V + \frac{1}{N} \sum_h W_h S_h^2} \quad (\text{Neyman})$$

The Neyman formula assumes equal costs and stratifies optimally.

6.8 4.8 Stratified sampling summary

Allocation	n_h	Best for
Proportional	$n \cdot N_h / N$	Self-weighting, simple
Neyman	$n \cdot N_h S_h / \sum N_j S_j$	Minimize variance, equal cost
Optimal cost	$n \cdot \frac{N_h S_h / \sqrt{c_h}}{\sum N_j S_j / \sqrt{c_j}}$	Different costs

Key insight: Stratification is never worse than SRS (when strata are correctly defined). It's better when between-stratum variance is large relative to within-stratum variance.

7 Part 5 — Systematic Sampling

7.1 5.1 What it is

- Order the population in some natural sequence.
- Choose a random start and then select every k -th unit.
- $k = N/n$ is the **sampling interval**.

7.2 5.2 Draw a systematic sample

```
# --- Self-contained: systematic sample ---
set.seed(501)
p5_N <- 500
p5_pop <- rnorm(p5_N, 50, 10)
p5_n <- 50
p5_k <- p5_N / p5_n

p5_start <- sample(1:p5_k, 1)
p5_idx <- seq(p5_start, by = p5_k, length.out = p5_n)
p5_sys <- p5_pop[p5_idx]

p5_ybar <- mean(p5_sys)
p5_s2 <- var(p5_sys)

round(c(ybar = p5_ybar, s2 = p5_s2, start = p5_start, k = p5_k), 4)

##      ybar      s2     start      k
## 52.4323 135.6649   6.0000 10.0000
```

7.3 5.3 Variance under systematic sampling

If the population is approximately random, use the SRS formula as an approximation:

$$\widehat{\text{Var}}(\bar{y}_{sys}) \approx \left(1 - \frac{n}{N}\right) \frac{s^2}{n}$$

The true variance depends on the correlation between units within a sampling interval.

Two common approximations: 1. **Treat as SRS:** conservative if the population has no periodic pattern.
2. **Successive-difference (variance among intervals):** accounts for ordering.

```
# --- Self-contained: systematic variance (successive-difference) ---
set.seed(502)
p5_N <- 500
p5_pop <- rnorm(p5_N, 50, 10)
p5_n <- 50
p5_k <- p5_N / p5_n
p5_start <- sample(1:p5_k, 1)
p5_idx <- seq(p5_start, by = p5_k, length.out = p5_n)
p5_sys <- p5_pop[p5_idx]

# Successive-difference estimator
p5_diffs <- diff(p5_sys)
p5_var_sd <- (1 - p5_n / p5_N) * sum(p5_diffs^2) / (2 * p5_n * (p5_n - 1))
p5_var_srs <- (1 - p5_n / p5_N) * var(p5_sys) / p5_n

round(c(var_sd = p5_var_sd, var_srs = p5_var_srs), 4)

##      var_sd var_srs
## 1.6194 1.6590
```

7.4 5.4 When systematic beats SRS

- Population is **ordered** by a variable correlated with y .

- Systematic sampling behaves like implicit stratification.
- **Danger:** periodic patterns (e.g., every 7th unit) can bias the estimate.

7.5 5.5 Systematic sampling summary

Property	Value
Selection	Every k -th unit after random start
π_i	$1/k$ for most units
Estimator of mean	$\bar{y}_{sys}$
Variance approximation	FPC $\cdot s^2/n$
When to use	Ordered frame, easy to implement
Risk	Periodicity in the frame

8 Part 6 — Cluster Sampling

8.1 6.1 What it is

- Population is divided into **clusters** (groups of units).
- **Sample clusters** (not individual units).
- Measure all units in sampled clusters.

Example: Sampling schools (clusters) and testing all students in selected schools.

8.2 6.2 Cluster sampling vs SRS

Advantages: - Cheaper (travel cost, setup cost). - Requires only a list of clusters, not units.

Disadvantage: - Higher variance if units within clusters are similar (intracluster correlation).

8.3 6.3 Build a clustered population

```
# --- Self-contained: clustered population ---
set.seed(601)
p6_N_clusters <- 50          # 50 clusters
p6_cluster_size <- 20        # 20 units each
p6_N_units <- p6_N_clusters * p6_cluster_size

p6_cluster_effect <- rnorm(p6_N_clusters, 0, 3)
p6_pop <- rep(p6_cluster_effect, each = p6_cluster_size) +
  rnorm(p6_N_units, 50, 5)
p6_cluster_id <- rep(1:p6_N_clusters, each = p6_cluster_size)

p6_mu <- mean(p6_pop)
round(p6_mu, 4)

## [1] 49.8371
```

8.4 6.4 One-stage cluster sampling

Sample m clusters out of M . Measure all units in each.

Estimator of mean:

$$\bar{y}_{cl} = \frac{\sum_{i \in S_{clusters}} \sum_{j=1}^{N_i} y_{ij}}{\sum_{i \in S_{clusters}} N_i}$$

Estimator of total:

$$\hat{\tau}_{cl} = \frac{M}{m} \sum_{i \in S} t_i, \quad t_i = \sum_j y_{ij}$$

```
# --- Self-contained: cluster sample ---
set.seed(602)
p6_N_clusters <- 50
p6_cluster_size <- 20
p6_cluster_effect <- rnorm(p6_N_clusters, 0, 3)
p6_pop <- rep(p6_cluster_effect, each = p6_cluster_size) +
  rnorm(p6_N_clusters * p6_cluster_size, 50, 5)
p6_cluster_id <- rep(1:p6_N_clusters, each = p6_cluster_size)
p6_mu <- mean(p6_pop)

p6_m <- 10 # sample 10 clusters
p6_sampled_clusters <- sample(1:p6_N_clusters, p6_m)

p6_totals <- sapply(p6_sampled_clusters, function(k) {
  sum(p6_pop[p6_cluster_id == k])
})
p6_sizes <- sapply(p6_sampled_clusters, function(k) {
  sum(p6_cluster_id == k)
})

# Estimator of mean (ratio estimator)
p6_ybar_cl <- sum(p6_totals) / sum(p6_sizes)

# Total estimator
p6_tau_hat <- p6_N_clusters * mean(p6_totals)

round(c(ybar_cl = p6_ybar_cl, tau_hat = p6_tau_hat), 4)

## ybar_cl tau_hat
## 47.781 47781.019
```

8.5 6.5 Variance under cluster sampling

$$\widehat{\text{Var}}(\hat{\tau}) = M^2 \left(1 - \frac{m}{M}\right) \frac{s_t^2}{m}$$

where s_t^2 is the variance of the cluster totals.

```
# --- Self-contained: cluster variance ---
set.seed(603)
p6_N_clusters <- 50
p6_cluster_size <- 20
p6_cluster_effect <- rnorm(p6_N_clusters, 0, 3)
p6_pop <- rep(p6_cluster_effect, each = p6_cluster_size) +
  rnorm(p6_N_clusters * p6_cluster_size, 50, 5)
p6_cluster_id <- rep(1:p6_N_clusters, each = p6_cluster_size)
```

```

p6_m <- 10
p6_sampled_clusters <- sample(1:p6_N_clusters, p6_m)
p6_totals <- sapply(p6_sampled_clusters, function(k) sum(p6_pop[p6_cluster_id == k]))

p6_tau_hat <- p6_N_clusters * mean(p6_totals)
p6_Var_tau <- p6_N_clusters^2 * (1 - p6_m / p6_N_clusters) *
  var(p6_totals) / p6_m

round(c(tau_hat = p6_tau_hat, se_tau = sqrt(p6_Var_tau)), 4)

##      tau_hat      se_tau
## 49972.7579   459.5315

```

8.6 6.6 Design effect for cluster sampling

$$\text{deff} = \frac{\text{Var}(\hat{\theta}_{\text{cluster}})}{\text{Var}(\hat{\theta}_{\text{SRS}})} = 1 + (N_i - 1)\rho$$

where ρ = **intraclass correlation** (similarity of units within clusters).

- $\rho = 0$: $\text{deff} = 1$ (clusters don't hurt).
- $\rho > 0$: $\text{deff} > 1$ (clusters hurt).
- $\rho = 1$: $\text{deff} = N_i$ (clusters as bad as n/N_i units).

```

# --- Self-contained: deff for clusters ---
set.seed(604)
p6_N_clusters <- 50
p6_cluster_size <- 20
p6_cluster_effect <- rnorm(p6_N_clusters, 0, 3)
p6_pop <- rep(p6_cluster_effect, each = p6_cluster_size) +
  rnorm(p6_N_clusters * p6_cluster_size, 50, 5)
p6_cluster_id <- rep(1:p6_N_clusters, each = p6_cluster_size)

# Intraclass correlation via ANOVA
p6_df <- tibble(y = p6_pop, cluster = factor(p6_cluster_id))
p6_aov <- aov(y ~ cluster, data = p6_df)
p6_msb <- summary(p6_aov)[[1]]["cluster", "Mean Sq"]
p6_msw <- summary(p6_aov)[[1]]["Residuals", "Mean Sq"]
p6_rho <- (p6_msb - p6_msw) / (p6_msb + (p6_cluster_size - 1) * p6_msw)
p6_deff <- 1 + (p6_cluster_size - 1) * p6_rho

round(c(rho = p6_rho, deff = p6_deff), 4)

##      rho      deff
## 0.2712  6.1522

```

8.7 6.7 Cluster sampling summary

Property	Value
Sample unit	Cluster
Estimator of total	$M\bar{t}$
Variance	Depends on cluster total variance
Design effect	$1 + (N_i - 1)\rho$
Best for	Geographically dispersed populations

Property	Value
Risk	High intraclass correlation inflates variance

9 Part 7 — Two-Stage Sampling

9.1 7.1 What it is

- **Stage 1:** Sample m primary sampling units (PSUs = clusters).
- **Stage 2:** Sample n_i units (SSUs) from each selected PSU.

Example: Sample schools, then sample students within schools.

9.2 7.2 Two-stage sample

```
# --- Self-contained: two-stage sample ---
set.seed(701)
p7_N_PSU <- 40
p7_N_SSU <- 25
p7_N <- p7_N_PSU * p7_N_SSU

p7_PSU_effect <- rnorm(p7_N_PSU, 0, 3)
p7_pop <- rep(p7_PSU_effect, each = p7_N_SSU) +
  rnorm(p7_N, 50, 5)
p7_PSU_id <- rep(1:p7_N_PSU, each = p7_N_SSU)
p7_mu <- mean(p7_pop)

# Stage 1: sample m PSUs
p7_m <- 8
p7_sampled_PSU <- sample(1:p7_N_PSU, p7_m)

# Stage 2: sample n SSUs from each PSU
p7_n <- 10
p7_sample <- unlist(lapply(p7_sampled_PSU, function(k) {
  idx <- which(p7_PSU_id == k)
  sample(idx, p7_n, replace = FALSE)
}))

p7_ybar_2stage <- mean(p7_pop[p7_sample])
round(c(mu = p7_mu, ybar = p7_ybar_2stage), 4)

##      mu      ybar
## 49.7045 50.8636
```

9.3 7.3 Variance

$$\widehat{\text{Var}}(\bar{y}_{2st}) = \left(1 - \frac{m}{M}\right) \frac{s_b^2}{m} + \frac{1}{m} \left(1 - \frac{n}{N_i}\right) \frac{s_w^2}{n}$$

where: - s_b^2 = between-PSU variance of PSU means. - s_w^2 = within-PSU variance.

```

# --- Self-contained: two-stage variance ---
set.seed(702)
p7_N_PSU <- 40; p7_N_SSU <- 25
p7_PSU_effect <- rnorm(p7_N_PSU, 0, 3)
p7_pop <- rep(p7_PSU_effect, each = p7_N_SSU) + rnorm(p7_N_PSU * p7_N_SSU, 50, 5)
p7_PSU_id <- rep(1:p7_N_PSU, each = p7_N_SSU)

p7_m <- 8
p7_sampled_PSU <- sample(1:p7_N_PSU, p7_m)
p7_n <- 10
p7_sample <- unlist(lapply(p7_sampled_PSU, function(k) {
  sample(which(p7_PSU_id == k), p7_n)
}))

p7_PSU_means <- sapply(p7_sampled_PSU, function(k) {
  mean(p7_pop[p7_PSU_id == k])
})
p7_PSU_vars <- sapply(p7_sampled_PSU, function(k) {
  var(p7_pop[p7_PSU_id == k])
})

p7_sb2 <- var(p7_PSU_means)
p7_sw2 <- mean(p7_PSU_vars)

p7_Var_2st <- (1 - p7_m / p7_N_PSU) * p7_sb2 / p7_m +
  (1 / p7_m) * (1 - p7_n / p7_N_SSU) * p7_sw2 / p7_n
p7_SE_2st <- sqrt(p7_Var_2st)

round(c(ybar_2st = mean(p7_pop[p7_sample]),
  var_2st = p7_Var_2st,
  se_2st = p7_SE_2st), 4)

## ybar_2st  var_2st  se_2st
## 47.7307   0.4302   0.6559

```

9.4 7.4 Two-stage summary

Property	Value
Stage 1	Sample m PSUs
Stage 2	Sample n_i SSUs per PSU
Estimator	$\bar{y}$ over all sampled units
Variance	Combines between- and within-PSU variance
Use when	Multi-level populations
Special case	$n_i = N_i \rightarrow$ cluster sampling

10 Part 8 — Ratio and Regression Estimators

10.1 8.1 Ratio estimator

When an auxiliary variable x is correlated with y and μ_x is known:

$$\hat{\mu}_R = \frac{\bar{y}}{\bar{x}} \mu_x$$

Bias: Approximate bias = $\frac{1}{n}(1 - n/N)(\bar{R}C_x^2 - \rho C_x C_y)$ where $\bar{R} = \mu_y/\mu_x$.

Variance approximation:

$$\widehat{\text{Var}}(\hat{\mu}_R) \approx \left(1 - \frac{n}{N}\right) \frac{s_d^2}{n}$$

where $d_i = y_i - \bar{R}x_i$ and $\bar{R} = \bar{y}/\bar{x}$.

```
# --- Self-contained: ratio estimator ---
set.seed(801)
p8_N <- 500
p8_x <- rnorm(p8_N, 20, 4)
p8_y <- 3 * p8_x + rnorm(p8_N, 0, 3)
p8_mu_x <- mean(p8_x)
p8_mu_y <- mean(p8_y)

p8_n <- 50
p8_srs <- sample(1:p8_N, p8_n)
p8_x_s <- p8_x[p8_srs]
p8_y_s <- p8_y[p8_srs]

p8_R_hat <- mean(p8_y_s) / mean(p8_x_s)
p8_mu_R <- p8_R_hat * p8_mu_x
p8_d <- p8_y_s - p8_R_hat * p8_x_s
p8_Var_R <- (1 - p8_n / p8_N) * var(p8_d) / p8_n

round(c(mu_y = p8_mu_y, mu_R = p8_mu_R, se_R = sqrt(p8_Var_R)), 4)

##      mu_y      mu_R      se_R
## 59.3910 58.8773  0.3973
```

When to use: x is strongly correlated with y , and μ_x is known.

10.2 8.2 Regression estimator

$$\hat{\mu}_{lr} = \bar{y} + b(\mu_x - \bar{x}), \quad b = \frac{s_{xy}}{s_x^2}$$

Variance:

$$\widehat{\text{Var}}(\hat{\mu}_{lr}) \approx \left(1 - \frac{n}{N}\right) \frac{s_r^2}{n}$$

where s_r^2 = residual variance from regressing y on x .

```
# --- Self-contained: regression estimator ---
set.seed(802)
p8_N <- 500
p8_x <- rnorm(p8_N, 20, 4)
p8_y <- 3 * p8_x + rnorm(p8_N, 0, 3)
p8_mu_x <- mean(p8_x)
p8_mu_y <- mean(p8_y)

p8_n <- 50
p8_srs <- sample(1:p8_N, p8_n)
```

```

p8_x_s <- p8_x[p8_srs]; p8_y_s <- p8_y[p8_srs]

p8_b <- cov(p8_x_s, p8_y_s) / var(p8_x_s)
p8_mu_lr <- mean(p8_y_s) + p8_b * (p8_mu_x - mean(p8_x_s))

p8_resid <- p8_y_s - (mean(p8_y_s) + p8_b * (p8_x_s - mean(p8_x_s)))
p8_Var_lr <- (1 - p8_n / p8_N) * var(p8_resid) * (p8_n - 1) /
  (p8_n - 2) / p8_n

round(c(mu_y = p8_mu_y, mu_lr = p8_mu_lr, se_lr = sqrt(p8_Var_lr)), 4)

##      mu_y      mu_lr      se_lr
## 59.5031 59.4367  0.4145

```

Ratio vs regression: Ratio assumes the line passes through the origin. Regression doesn't.

10.3 8.3 Reporting

“With auxiliary variable x known (mean $\mu_x = 20.1$), the ratio estimator gives $\hat{\mu}_R = 60.2$ with SE 0.85, compared to the SRS estimator $\bar{y} = 60.5$ with SE 1.12. The ratio estimator is more efficient because it uses the correlation between x and y .”

11 Part 9 — Bias, MSE, and Relative Efficiency

11.1 9.1 Simulate bias, variance, MSE

```

# --- Self-contained: bias/variance/MSE simulation ---
set.seed(901)
p9_N <- 1000
p9_pop <- rnorm(p9_N, 50, 10)
p9_mu <- mean(p9_pop)

p9_n <- 30
p9_B <- 3000

p9_means <- numeric(p9_B)
p9_medians <- numeric(p9_B)

for (b in 1:p9_B) {
  s <- sample(p9_pop, p9_n)
  p9_means[b] <- mean(s)
  p9_medians[b] <- median(s)
}

p9_tab <- tibble(
  Estimator = c("Mean", "Median"),
  Bias = c(mean(p9_means) - p9_mu,
            mean(p9_medians) - p9_mu),
  Variance = c(var(p9_means), var(p9_medians)),
  MSE = c(mean((p9_means - p9_mu)^2),
           mean((p9_medians - p9_mu)^2))
)

```

```
)
kable(p9_tab, digits = 4)
```

Estimator	Bias	Variance	MSE
Mean	-0.0559	3.1887	3.1908
Median	0.1770	5.1810	5.2106

Interpretation: - Mean is unbiased (bias = 0). - Median may be biased but often has smaller variance if the population is skewed. - Compare MSE to choose.

11.2 9.2 Relative efficiency

```
# --- Self-contained: relative efficiency ---
set.seed(902)
p9_N <- 1000
p9_pop <- rnorm(p9_N, 50, 10)
p9_mu <- mean(p9_pop)
p9_n <- 30; p9_B <- 3000

p9_means <- replicate(p9_B, mean(sample(p9_pop, p9_n)))
p9_medians <- replicate(p9_B, median(sample(p9_pop, p9_n)))

p9_RE_median_vs_mean <- var(p9_medians) / var(p9_means)
p9_RE_mean_vs_median <- var(p9_means) / var(p9_medians)

round(c(RE_median_vs_mean = p9_RE_median_vs_mean,
        RE_mean_vs_median = p9_RE_mean_vs_median), 4)

## RE_median_vs_mean RE_mean_vs_median
##                1.3838                0.7226
```

Interpretation: RE > 1 → numerator is more efficient (lower variance).

11.3 9.3 Bias of the ratio estimator (empirical)

```
# --- Self-contained: ratio estimator bias ---
set.seed(903)
p9_N <- 500
p9_x <- rnorm(p9_N, 20, 4)
p9_y <- 3 * p9_x + rnorm(p9_N, 0, 3)
p9_mu_x <- mean(p9_x)
p9_mu_y <- mean(p9_y)

p9_n <- 30; p9_B <- 3000
p9_R_est <- replicate(p9_B, {
  idx <- sample(1:p9_N, p9_n)
  mean(p9_y[idx]) / mean(p9_x[idx]) * p9_mu_x
})

round(c(true = p9_mu_y,
        mean_R = mean(p9_R_est),
        bias = mean(p9_R_est) - p9_mu_y,
```

```
var_R = var(p9_R_est),
mse_R = mean((p9_R_est - p9_mu_y)^2), 4)
```

```
##      true  mean_R    bias   var_R   mse_R
## 60.9217 60.9176 -0.0041  0.2922  0.2921
```

11.4 9.4 Comparing estimators summary

Criterion	Formula	Decision
Bias	$E[\hat{\theta}] - \theta$	Closer to 0 is better
Variance	$E[(\hat{\theta} - E\hat{\theta})^2]$	Smaller is better
MSE	Bias ² + Variance	Smaller is better
RE of $\hat{\theta}_1$ vs $\hat{\theta}_2$	$MSE(2)/MSE(1)$	$> 1 \rightarrow \hat{\theta}_1$ better

12 Part 10 — Coverage Probability Simulations

12.1 10.1 Coverage of a 95% CI for μ (SRS)

```
# --- Self-contained: CI coverage ---
set.seed(1001)
p10_N <- 500
p10_pop <- rnorm(p10_N, 50, 10)
p10_mu <- mean(p10_pop)

p10_n <- 30
p10_B <- 2000
p10_z <- qnorm(0.975)

p10_cov <- logical(p10_B)
p10_len <- numeric(p10_B)

for (b in 1:p10_B) {
  s <- sample(p10_pop, p10_n)
  ybar <- mean(s); s2 <- var(s)
  se <- sqrt((1 - p10_n / p10_N) * s2 / p10_n)
  ci <- c(ybar - p10_z * se, ybar + p10_z * se)
  p10_cov[b] <- (p10_mu >= ci[1]) & (p10_mu <= ci[2])
  p10_len[b] <- ci[2] - ci[1]
}

round(c(coverage = mean(p10_cov), mean_length = mean(p10_len)), 4)

##      coverage mean_length
##      0.9420      6.8114
```

Interpretation: With FPC, coverage should be near 0.95. This validates the CI formula.

12.2 10.2 Coverage with FPC vs without

```

# --- Self-contained: coverage with and without FPC ---
set.seed(1002)
p10_N <- 500
p10_pop <- rnorm(p10_N, 50, 10)
p10_mu <- mean(p10_pop)
p10_n <- 50 # 10% sampling fraction
p10_B <- 2000
p10_z <- qnorm(0.975)

p10_cov_fpc <- logical(p10_B)
p10_cov_nofpc <- logical(p10_B)
p10_len_fpc <- numeric(p10_B)
p10_len_nofpc <- numeric(p10_B)

for (b in 1:p10_B) {
  s <- sample(p10_pop, p10_n)
  ybar <- mean(s); s2 <- var(s)
  se_fpc <- sqrt((1 - p10_n / p10_N) * s2 / p10_n)
  se_nf <- sqrt(s2 / p10_n)

  ci_f <- c(ybar - p10_z * se_fpc, ybar + p10_z * se_fpc)
  ci_n <- c(ybar - p10_z * se_nf, ybar + p10_z * se_nf)

  p10_cov_fpc[b] <- (p10_mu >= ci_f[1]) & (p10_mu <= ci_f[2])
  p10_cov_nofpc[b] <- (p10_mu >= ci_n[1]) & (p10_mu <= ci_n[2])
  p10_len_fpc[b] <- ci_f[2] - ci_f[1]
  p10_len_nofpc[b] <- ci_n[2] - ci_n[1]
}

tibble(
  CI = c("With FPC", "Without FPC"),
  Coverage = round(c(mean(p10_cov_fpc), mean(p10_cov_nofpc)), 4),
  MeanLength = round(c(mean(p10_len_fpc), mean(p10_len_nofpc)), 4)
) %>% kable()

```

CI	Coverage	MeanLength
With FPC	0.9385	4.9205
Without FPC	0.9540	5.1867

Interpretation: The FPC CI is shorter but should have similar coverage. If the sampling fraction is small, both are close.

12.3 10.3 Coverage under misspecified distribution

```

# --- Self-contained: coverage when population is skewed ---
set.seed(1003)
p10_N <- 500
p10_pop <- rexp(p10_N, 1) # skewed
p10_mu <- mean(p10_pop)

p10_n <- 30

```

```

p10_B <- 2000
p10_z <- qnorm(0.975)

p10_cov <- numeric(p10_B)
for (b in 1:p10_B) {
  s <- sample(p10_pop, p10_n)
  se <- sqrt((1 - p10_n / p10_N) * var(s) / p10_n)
  p10_cov[b] <- (p10_mu >= mean(s) - p10_z * se) &
    (p10_mu <= mean(s) + p10_z * se)
}
mean(p10_cov)

## [1] 0.9105

```

Interpretation: For skewed populations, coverage can be lower than 0.95 for small n — the CLT hasn't kicked in yet.

13 Part 11 — Sample Size Determination

13.1 11.1 For a target variance of the mean

$$n = \frac{S^2}{V + S^2/N}$$

where V = target variance of $\bar{y}$.

```

# --- Self-contained: sample size for target variance ---
p11_N <- 1000
p11_S2 <- 100
p11_V <- 1      # target variance of ybar

p11_n <- p11_S2 / (p11_V + p11_S2 / p11_N)
ceiling(p11_n)

## [1] 91

```

13.2 11.2 For a target margin of error (with FPC)

$$n_0 = \frac{z^2 S^2}{e^2}, \quad n = \frac{n_0}{1 + n_0/N}$$

```

# --- Self-contained: sample size for margin of error ---
p11_N <- 1000
p11_S2 <- 100
p11_e <- 2
p11_z <- qnorm(0.975)

p11_n0 <- p11_z^2 * p11_S2 / p11_e^2
p11_n <- ceiling(p11_n0 / (1 + p11_n0 / p11_N))
p11_n

## [1] 88

```

13.3 11.3 For a proportion

$$n_0 = \frac{z^2 p(1-p)}{e^2}, \quad n = \frac{n_0}{1 + n_0/N}$$

Use $p = 0.5$ if unknown (worst case).

```
# --- Self-contained: proportion sample size ---
p11_N <- 2000
p11_p <- 0.5 # conservative
p11_e <- 0.05
p11_z <- qnorm(0.975)

p11_n0 <- p11_z^2 * p11_p * (1 - p11_p) / p11_e^2
p11_n <- ceiling(p11_n0 / (1 + p11_n0 / p11_N))
p11_n
```

```
## [1] 323
```

13.4 11.4 Stratified sample size for target variance (proportional)

$$n = \frac{\sum_h W_h^2 S_h^2 / w_h}{V + \frac{1}{N} \sum_h W_h S_h^2}$$

where $w_h = n_h/n$ is the fraction allocated to stratum h .

For proportional allocation ($w_h = W_h$):

$$n = \frac{\sum_h W_h S_h^2}{V + \frac{1}{N} \sum_h W_h S_h^2}$$

```
# --- Self-contained: stratified sample size ---
p11_N_h <- c(200, 400, 400)
p11_S_h <- c(5, 8, 12)
p11_N <- sum(p11_N_h)
p11_W_h <- p11_N_h / p11_N
p11_V <- 1

p11_numerator <- sum(p11_W_h * p11_S_h^2)
p11_denominator <- p11_V + (1 / p11_N) * sum(p11_W_h * p11_S_h^2)
p11_n <- ceiling(p11_numerator / p11_denominator)
p11_n
```

```
## [1] 82
```

13.5 11.5 Sample size for Neyman allocation

$$n = \frac{(\sum_h W_h S_h)^2}{V + \frac{1}{N} \sum_h W_h S_h^2}$$

```
# --- Self-contained: Neyman sample size ---
p11_N_h <- c(200, 400, 400)
p11_S_h <- c(5, 8, 12)
p11_N <- sum(p11_N_h)
p11_W_h <- p11_N_h / p11_N
p11_V <- 1
```

```
p11_num <- (sum(p11_W_h * p11_S_h))^2
p11_den <- p11_V + (1 / p11_N) * sum(p11_W_h * p11_S_h^2)
p11_n <- ceiling(p11_num / p11_den)
p11_n

## [1] 75
```

13.6 11.6 Sample size summary

Goal	Formula
Target variance V	$n = S^2 / (V + S^2 / N)$
Target margin e	$n = z^2 S^2 / e^2$, adjust with FPC
Proportion	$n = z^2 p(1 - p) / e^2$, adjust
Stratified proportional	$n = \frac{\sum W_h S_h^2}{V + \frac{1}{N} \sum W_h S_h^2}$
Stratified Neyman	$n = \frac{(\sum W_h S_h)^2}{V + \frac{1}{N} \sum W_h S_h^2}$

14 Part 12 — Design Effects

14.1 12.1 Definition

$$\text{deff} = \frac{\text{Var}(\hat{\theta}_{\text{design}})}{\text{Var}(\hat{\theta}_{\text{SRS}})}$$

- $\text{deff} = 1$: same efficiency as SRS.
- $\text{deff} > 1$: less efficient than SRS.
- $\text{deff} < 1$: more efficient than SRS (stratification does this).

14.2 12.2 Effective sample size

$$n_{\text{eff}} = \frac{n}{\text{deff}}$$

If you have a cluster sample with $\text{deff} = 3$, your 300 observations give the same precision as 100 SRS observations.

14.3 12.3 deff for stratified sampling

$$\text{deff} = \frac{\sum_h W_h^2 S_h^2 / n_h}{S^2 / n}$$

With proportional allocation, $\text{deff} = 1$ always.

```
# --- Self-contained: deff for stratification ---
set.seed(1201)
p12_N_h <- c(200, 400, 400)
p12_S_h <- c(5, 8, 12)
p12_N <- sum(p12_N_h)
p12_W_h <- p12_N_h / p12_N
p12_S2 <- sum(p12_W_h * p12_S_h^2) +
  sum(p12_W_h * (c(10, 15, 20) - sum(p12_W_h * c(10, 15, 20)))^2)
```

```

p12_n    <- 60
p12_n_h  <- round(p12_n * p12_W_h)

p12_var_st <- sum(p12_W_h^2 * p12_S_h^2 / p12_n_h)
p12_var_srs <- p12_S2 / p12_n

p12_deff <- p12_var_st / p12_var_srs
round(p12_deff, 4)

## [1] 0.863

```

14.4 12.4 deff for cluster sampling

$$\text{deff} = 1 + (N_i - 1)\rho$$

Already covered in Part 6.6.

14.5 12.5 Reporting

“The stratified design has a design effect of 0.62, so it is 1.6 times more efficient than SRS. The cluster design has $\text{deff} = 3.1$ due to high intraclass correlation ($\rho = 0.22$), meaning 300 clustered units give the same precision as 100 SRS units.”

15 Part 13 — Horvitz-Thompson Estimator

15.1 13.1 Definition

For any sampling design with inclusion probabilities π_i :

$$\hat{\tau}_{HT} = \sum_{i \in S} \frac{y_i}{\pi_i}$$

This is **unbiased** for the population total τ .

For the mean:

$$\hat{\mu}_{HT} = \frac{\hat{\tau}_{HT}}{N}$$

15.2 13.2 Variance of HT estimator

$$\text{Var}(\hat{\tau}_{HT}) = \sum_{i=1}^N \sum_{j=1}^N (\pi_{ij} - \pi_i \pi_j) \frac{y_i}{\pi_i} \frac{y_j}{\pi_j}$$

where π_{ij} = joint inclusion probability of units i and j .

Unbiased variance estimator (Yates-Grundy):

$$\widehat{\text{Var}}(\hat{\tau}_{HT}) = \sum_{i < j \in S} \frac{\pi_i \pi_j - \pi_{ij}}{\pi_{ij}} \left(\frac{y_i}{\pi_i} - \frac{y_j}{\pi_j} \right)^2$$

15.3 13.3 For SRS

$\pi_i = n/N$, $\pi_{ij} = n(n-1)/(N(N-1))$.

$$\hat{\tau}_{HT} = \frac{N}{n} \sum_{i \in S} y_i = N\bar{y}$$

Same as the usual estimator.

```
# --- Self-contained: HT for SRS ---
set.seed(1301)
p13_N <- 500
p13_pop <- rnorm(p13_N, 50, 10)
p13_n <- 50

p13_srs <- sample(1:p13_N, p13_n)
p13_pi <- p13_n / p13_N
p13_tau_hat <- sum(p13_pop[p13_srs] / p13_pi)

round(c(tau_hat = p13_tau_hat, N_ybar = p13_N * mean(p13_pop[p13_srs])), 4)

## tau_hat    N_ybar
## 25601.05 25601.05
```

15.4 13.4 Unequal probability sampling

When units have different π_i (e.g., PPS), the HT estimator gives an unbiased estimate even though units are not equally weighted.

```
# --- Self-contained: HT with unequal probabilities ---
set.seed(1302)
p13_N <- 10
p13_y <- c(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
p13_size <- p13_y # size proportional to y
p13_n <- 3

# Sample with probabilities proportional to size
p13_srs <- sample(1:p13_N, p13_n,
                  prob = p13_size / sum(p13_size),
                  replace = FALSE)

# Inclusion probabilities (approximate for PPS without replacement)
p13_pi <- p13_n * p13_size / sum(p13_size)
p13_tau_hat <- sum(p13_y[p13_srs] / p13_pi[p13_srs])

round(c(true_tau = sum(p13_y), HT = p13_tau_hat), 4)

## true_tau    HT
##      550    550
```

15.5 13.5 HT summary

Property	Value
Estimator	$\hat{\tau} = \sum_{i \in S} y_i / \pi_i$
Bias	Unbiased for any design

Property	Value
Variance	Depends on joint inclusion probabilities
When to use	Design-based inference for complex designs
Special case	SRS, stratified: reduces to standard estimators

16 Part 14 — Probability Proportional to Size (PPS)

16.1 14.1 What it is

Sample units with probability proportional to a **size measure** x_i . Larger units are more likely to be selected.

When to use: Businesses (size = employees), schools (size = enrollment), hospitals (size = beds).

16.2 14.2 PPS with replacement

Sample n units with probability $\propto x_i$. Since sizes differ, HT estimator simplifies.

Hansen-Hurwitz estimator of total:

$$\hat{\tau}_{HH} = \frac{1}{n} \sum_{i=1}^n \frac{y_i}{p_i}, \quad p_i = \frac{x_i}{\sum_j x_j}$$

```
# --- Self-contained: PPS with replacement ---
set.seed(1401)
p14_N <- 20
p14_x <- runif(p14_N, 10, 100) # size
p14_y <- 5 * p14_x + rnorm(p14_N, 0, 20)
p14_tau <- sum(p14_y)

p14_n <- 5
p14_idx <- sample(1:p14_N, p14_n,
                 prob = p14_x / sum(p14_x),
                 replace = TRUE)
p14_p <- p14_x / sum(p14_x)
p14_tau_hat <- mean(p14_y[p14_idx] / p14_p[p14_idx])

round(c(true_tau = p14_tau, HH = p14_tau_hat), 4)

## true_tau      HH
## 4673.583 4589.537
```

16.3 14.3 Variance of HH estimator

$$\widehat{\text{Var}}(\hat{\tau}_{HH}) = \frac{1}{n(n-1)} \sum_{i=1}^n \left(\frac{y_i}{p_i} - \hat{\tau}_{HH} \right)^2$$

```
# --- Self-contained: PPS variance ---
set.seed(1402)
p14_N <- 20
p14_x <- runif(p14_N, 10, 100)
p14_y <- 5 * p14_x + rnorm(p14_N, 0, 20)
p14_n <- 5
```

```

p14_idx <- sample(1:p14_N, p14_n, prob = p14_x / sum(p14_x), replace = TRUE)
p14_p <- p14_x / sum(p14_x)

p14_z <- p14_y[p14_idx] / p14_p[p14_idx]
p14_tau_hat <- mean(p14_z)
p14_Var_HH <- var(p14_z) / p14_n
p14_SE_HH <- sqrt(p14_Var_HH)

round(c(HH = p14_tau_hat, se = p14_SE_HH), 4)

##           HH           se
## 5568.4073    80.5728

```

16.4 14.4 PPS summary

Property	Value
Selection	Probability $\propto x_i$
Estimator of total	$\frac{1}{n} \sum y_i/p_i$
Variance	$\text{Var}(\bar{z})/n$ where $z_i = y_i/p_i$
Best for	Populations with wide size variation
Requires	Known size measure for all units

17 Part 15 — Non-Response

17.1 15.1 Types of non-response

- **Unit non-response:** The whole unit doesn't respond.
- **Item non-response:** The unit responds but skips some questions.

17.2 15.2 Effects on estimates

If non-responders differ systematically from responders, the estimate is **biased**:

$$\text{Bias} = \frac{\text{NR}}{N}(\bar{y}_{NR} - \bar{y}_R)$$

where NR = non-responders, R = responders.

```

# --- Self-contained: non-response bias ---
set.seed(1501)
p15_N <- 1000
p15_y <- c(rnorm(700, 50, 10),          # responders-to-be
          rnorm(300, 60, 10))          # non-responders-to-be
p15_truth <- mean(p15_y)

# Suppose 40% respond
p15_n_resp <- 400
p15_sample_resp <- sample(1:700, p15_n_resp) # only from responders

p15_est <- mean(p15_y[p15_sample_resp])

```

```
round(c(truth = p15_truth, estimate = p15_est,
        bias = p15_est - p15_truth), 4)
```

```
##      truth estimate      bias
## 53.5190 50.6411 -2.8780
```

17.3 15.3 Weighting adjustment

Give responders weights inversely proportional to their response probability.

Weight = 1 / (response rate) for each adjustment cell.

```
# --- Self-contained: weighted estimate ---
set.seed(1502)
p15_N <- 1000
p15_group <- sample(c("A", "B"), p15_N, replace = TRUE)
p15_y <- ifelse(p15_group == "A", rnorm(p15_N, 50, 10), rnorm(p15_N, 60, 10))

# Response rates differ by group
p15_resp_prob <- ifelse(p15_group == "A", 0.8, 0.4)
p15_responded <- runif(p15_N) < p15_resp_prob
p15_sample <- which(p15_responded)

# Unweighted (biased)
p15_unw <- mean(p15_y[p15_sample])

# Weighted by 1/response-prob
p15_w <- 1 / p15_resp_prob[p15_sample]
p15_wtd <- sum(p15_y[p15_sample] * p15_w) / sum(p15_w)

round(c(truth = mean(p15_y),
        unweighted = p15_unw,
        weighted = p15_wtd), 4)
```

```
##      truth unweighted  weighted
## 54.9755    53.2413    54.8074
```

17.4 15.4 Summary

Type	Effect	Adjustment
MCAR	Small bias	Ignore or drop
MAR	Moderate bias	Weighting, imputation
MNAR	Large bias	Model non-response, sensitivity analysis

Always report: response rate, potential bias.

18 Part 16 — Bootstrap Variance Estimation

18.1 16.1 Why bootstrap?

For complex designs (cluster, two-stage, PPS), the theoretical variance may be hard to derive. Bootstrap gives an empirical estimate by resampling.

18.2 16.2 SRS bootstrap

Resample with replacement from the sample, compute the statistic, repeat B times. The variance of the bootstrap replicates approximates the sampling variance.

```
# --- Self-contained: bootstrap variance for SRS ---
set.seed(1601)
p16_N <- 500
p16_pop <- rnorm(p16_N, 50, 10)
p16_n <- 50
p16_srs <- sample(p16_pop, p16_n)

p16_B <- 2000
p16_boot_means <- replicate(p16_B, {
  s <- sample(p16_srs, p16_n, replace = TRUE)
  mean(s)
})

p16_boot_var <- var(p16_boot_means)
p16_theory_var <- (1 - p16_n / p16_N) * var(p16_srs) / p16_n

round(c(theory = p16_theory_var, bootstrap = p16_boot_var), 4)

##      theory bootstrap
##      1.8975      2.1039
```

Note: Standard bootstrap doesn't account for FPC. For design-based bootstrap, use a scaled version or a resampling scheme that respects the design.

18.3 16.3 Bootstrap CI

Percentile CI: the 2.5% and 97.5% quantiles of the bootstrap distribution.

```
# --- Self-contained: bootstrap CI ---
set.seed(1602)
p16_pop <- rnorm(500, 50, 10)
p16_n <- 50
p16_srs <- sample(p16_pop, p16_n)

p16_B <- 2000
p16_boot_means <- replicate(p16_B, mean(sample(p16_srs, p16_n, replace = TRUE)))

p16_ci_pct <- quantile(p16_boot_means, c(0.025, 0.975))

# Basic bootstrap CI
p16_ci_basic <- 2 * mean(p16_srs) - rev(p16_ci_pct)

round(c(percentile_lo = p16_ci_pct[1],
        percentile_hi = p16_ci_pct[2],
        basic_lo = p16_ci_basic[1],
        basic_hi = p16_ci_basic[2]), 4)

## percentile_lo.2.5% percentile_hi.97.5%      basic_lo.97.5%      basic_hi.2.5%
##              48.1894              54.4584              48.1502              54.4191
```

18.4 16.4 Cluster bootstrap

For cluster sampling, **resample clusters** (not units):

```
# --- Self-contained: cluster bootstrap ---
set.seed(1603)
p16_N_clusters <- 50
p16_cluster_size <- 20
p16_cluster_effect <- rnorm(p16_N_clusters, 0, 3)
p16_pop <- rep(p16_cluster_effect, each = p16_cluster_size) +
  rnorm(p16_N_clusters * p16_cluster_size, 50, 5)
p16_cluster_id <- rep(1:p16_N_clusters, each = p16_cluster_size)

p16_m <- 10
p16_sampled_clusters <- sample(1:p16_N_clusters, p16_m)

# Bootstrap: resample clusters
p16_B <- 2000
p16_boot_totals <- replicate(p16_B, {
  boot_clusters <- sample(p16_sampled_clusters, replace = TRUE)
  sum(sapply(boot_clusters, function(k) sum(p16_pop[p16_cluster_id == k])))
})

# Estimator: total
p16_totals <- sapply(p16_sampled_clusters, function(k) sum(p16_pop[p16_cluster_id == k]))
p16_tau_hat <- p16_N_clusters * mean(p16_totals)
p16_boot_se <- sd(p16_boot_totals) / sqrt(p16_m)

round(c(tau_hat = p16_tau_hat, se_theory = sqrt(p16_N_clusters^2 *
  (1 - p16_m / p16_N_clusters) *
  var(p16_totals) / p16_m),
  se_boot = p16_boot_se), 4)

##      tau_hat  se_theory    se_boot
## 48812.9736 1406.2434    96.0116
```

18.5 16.5 Bootstrap summary

Design	Resample
SRS	Units with replacement
Stratified	Within each stratum
Cluster	Whole clusters with replacement
Two-stage	PSUs, then SSUs within

19 Part 17 — Theory Q&A

Q: What is the difference between a parameter and a statistic? A: A parameter is a fixed (unknown) population value. A statistic is a random variable computed from a sample that estimates the parameter.

Q: What is the finite population correction and why does it matter? A: $FPC = 1 - n/N$. It reduces the variance of the sample mean when the sample is a large fraction of the population. Ignoring it overestimates the variance.

Q: What is the difference between sampling with and without replacement? A: With replacement, units can repeat; without replacement, each unit appears at most once. SRS usually means without replacement.

Q: What is the bias of the sample mean under SRS? A: Zero. Under SRS, $E[\bar{y}] = \mu$.

Q: What is the variance of the sample mean under SRS? A: $(1 - n/N)S^2/n$.

Q: What is stratified sampling and why use it? A: Divide the population into strata, sample independently within each. Use it to reduce variance by ensuring all subgroups are represented, and to control the sample size per stratum.

Q: What is proportional allocation? A: Sample size per stratum is proportional to stratum size: $n_h = nN_h/N$.

Q: What is Neyman allocation? A: $n_h = nN_hS_h/\sum_j N_jS_j$. Minimizes variance for fixed n with equal costs.

Q: What is optimal allocation with costs? A: $n_h = n \frac{N_hS_h/\sqrt{c_h}}{\sum_j N_jS_j/\sqrt{c_j}}$. Accounts for different per-unit costs across strata.

Q: When does stratification beat SRS? A: Always (weakly). Strictly when between-stratum variance is large relative to within-stratum variance.

Q: What is systematic sampling and when is it dangerous? A: Select every k -th unit after a random start. Dangerous when the frame has periodic patterns that align with k .

Q: What is cluster sampling? A: Sample whole groups (clusters) and measure all units within each sampled cluster. Used when travel/setup cost dominates.

Q: What is the design effect? A: $\text{deff} = \text{Var}(\hat{\theta}_{\text{design}})/\text{Var}(\hat{\theta}_{\text{SRS}})$. Values > 1 mean the design is less efficient than SRS.

Q: What is the intraclass correlation? A: The correlation between units within the same cluster. High ρ means clusters are homogeneous, so cluster sampling is inefficient.

Q: What is two-stage sampling? A: Sample PSUs (clusters) at stage 1, then SSUs within each selected PSU at stage 2. More flexible than one-stage cluster sampling.

Q: What is the Horvitz-Thompson estimator? A: $\hat{\tau}_{HT} = \sum_{i \in S} y_i/\pi_i$. Unbiased for the population total for any design, using inclusion probabilities.

Q: What is PPS sampling? A: Sample units with probability proportional to a size measure x_i . Efficient when sizes vary widely.

Q: What is the ratio estimator? A: $\hat{\mu}_R = (\bar{y}/\bar{x})\mu_x$. Uses a known auxiliary mean μ_x to improve precision.

Q: What is the regression estimator? A: $\hat{\mu}_{lr} = \bar{y} + b(\mu_x - \bar{x})$. Uses the linear regression of y on x to adjust the estimate.

Q: What is the difference between ratio and regression estimators? A: Ratio assumes the line passes through the origin; regression doesn't. Regression is more general but requires more computation.

Q: When is the median a better estimator than the mean? A: When the population is skewed or has outliers. The median is robust to extreme values.

Q: What is the coverage probability? A: The proportion of confidence intervals that contain the true parameter over repeated sampling. A 95% CI should have coverage 0.95.

Q: What is the difference between design-based and model-based inference? A: Design-based: the population is fixed, randomness comes from sampling. Model-based: the population is random, drawn from a superpopulation.

Q: What is the effective sample size? A: $n_{\text{eff}} = n/\text{deff}$. The size of an SRS sample that would give the same precision.

Q: How do you handle non-response? A: Report the response rate, check if non-response is related to the outcome (MCAR/MAR/MNAR), use weighting or imputation.

Q: Why does clustering inflate variance? A: Because units within a cluster tend to be similar, they provide less independent information than N_i separate SRS units.

20 Part 18 — Exam Scenarios

20.1 18.1 “Estimate the population mean under SRS”

1. Sample n units.
2. Compute $\bar{y} = \text{mean}(s)$.
3. Compute $\hat{V} = (1 - n/N)s^2/n$.
4. Build CI: $\bar{y} \pm z\sqrt{\hat{V}}$.
5. Report estimate + SE + CI.

20.2 18.2 “Compare SRS and stratified variance”

1. Compute theoretical variance of SRS.
2. Compute theoretical variance of stratified (with the allocation).
3. Compare ratio: $\text{Var}_{st}/\text{Var}_{SRS}$.
4. Report which design is more efficient.

20.3 18.3 “Optimal allocation”

1. List N_h and S_h for each stratum.
2. Compute $n_h = nN_hS_h/\sum N_jS_j$.
3. Round to integers, ensure total = n .
4. Compute variance with the resulting allocation.

20.4 18.4 “Bias and MSE of an estimator”

1. Simulate B samples.
2. Compute the estimator for each.
3. Bias = $\text{mean}(\text{estimates}) - \text{truth}$.
4. Variance = $\text{var}(\text{estimates})$.
5. MSE = $\text{mean}((\text{estimates} - \text{truth})^2)$.

20.5 18.5 “Coverage of a CI”

1. Simulate B samples.
2. For each, compute the CI.
3. Count how many CIs contain the true parameter.
4. Report coverage.

20.6 18.6 “Sample size for target precision”

1. Identify target (variance, margin of error, or coverage).
2. Get S^2 from pilot or guess.
3. Compute $n_0 = z^2S^2/e^2$.

4. Adjust with FPC: $n = n_0 / (1 + n_0 / N)$.
5. Round up.

20.7 18.7 “Design effect”

1. Compute variance under your design.
2. Compute variance under SRS with same n .
3. $\text{deff} = V_{\text{design}} / V_{\text{SRS}}$.
4. $n_{\text{eff}} = n / \text{deff}$.

20.8 18.8 “Cluster sampling”

1. Compute cluster totals.
2. HT or HH estimator.
3. Variance from cluster totals.
4. Design effect from intraclass correlation.

20.9 18.9 “Ratio/regression estimator”

1. Compute $\bar{y}$ and $\bar{x}$ from the sample.
2. Use known μ_x to compute $\hat{\mu}_R$ or $\hat{\mu}_{lr}$.
3. Compute residuals, then variance.
4. Compare to SRS.

20.10 18.10 “Non-response adjustment”

1. Identify response rates by group.
2. Compute weights = 1/response probability.
3. Compute weighted estimate.
4. Report difference from unweighted.

21 Part 19 — Quick Reference

Task	Command
SRS sample	<code>sample(pop, n, replace = FALSE)</code>
Sample with replacement	<code>sample(pop, n, replace = TRUE)</code>
Weighted sample	<code>sample(pop, n, prob = w, replace = TRUE)</code>
Sample mean	<code>mean(s)</code>
Sample variance	<code>var(s)</code>
FPC	<code>(1 - n/N)</code>
Var of sample mean	<code>(1 - n/N) * var(s) / n</code>
Total estimate	<code>N * mean(s)</code>
Var of total	<code>N^2 * (1 - n/N) * var(s) / n</code>
Proportion	<code>mean(binary_s)</code>
Prop variance	<code>(1 - n/N) * p * (1-p) / (n-1)</code>
Stratified mean	<code>sum(W_h * Xbar_h)</code>
Stratified var	<code>sum(W_h^2 * (1 - n_h/N_h) * s2_h / n_h)</code>
Proportional allocation	<code>n_h = round(n * N_h / N)</code>
Neyman allocation	<code>n_h = round(n * N_h * S_h / sum(N_h * S_h))</code>
Optimal with costs	<code>n_h = round(n * (N_h*S_h/sqrt(c_h)) / sum(...))</code>

Task	Command
Systematic sample	<code>idx = seq(start, by = k, length.out = n)</code>
Successive-difference var	<code>sum(diff(s)^2) / (2*n*(n-1)) * (1 - n/N)</code>
Cluster estimator	ratio of sums (<code>y_total / size_total</code>)
Cluster variance	<code>M^2 * (1 - m/M) * var(cluster_totals) / m</code>
Two-stage var	between + within components
Intraclass correlation	<code>(MSB - MSW) / (MSB + (n_i - 1)*MSW)</code>
Design effect	<code>deff = Var_design / Var_SRS</code>
Effective sample size	<code>n_eff = n / deff</code>
Ratio estimator	<code>mean(y) / mean(x) * mu_x</code>
Regression estimator	<code>mean(y) + b * (mu_x - mean(x))</code>
HT estimator (total)	<code>sum(y_i / pi_i)</code>
PPS-HH estimator	<code>mean(y_i / p_i)</code>
Bias	<code>mean(estimated) - truth</code>
Variance	<code>var(estimated)</code>
MSE	<code>mean((estimated - truth)^2)</code>
Relative efficiency	<code>MSE_2 / MSE_1</code>
Coverage	<code>mean(truth >= ci_lo & truth <= ci_hi)</code>
Sample size for margin	<code>z^2 * S^2 / e^2</code> , adjust with FPC
Sample size for proportion	<code>z^2 * p * (1-p) / e^2</code> , adjust
Sample size for target var	<code>S^2 / (V + S^2/N)</code>
Bootstrap CI	<code>quantile(boot_est, c(0.025, 0.975))</code>
Cluster bootstrap	resample whole clusters

22 Part 20 — Common Mistakes

1. **Forgetting** `replace = FALSE` for SRS without replacement.
2. **Using** `sd(s)` instead of `var(s)` in variance formulas.
3. **Confusing** N and n — population vs sample size.
4. **Forgetting the FPC** when the sample is a large fraction of N .
5. **Not setting** `set.seed()` in simulations — results change.
6. **Using theoretical formulas** when the question asks for empirical estimates.
7. **Confusing bias and variance.**
8. **Rounding** n_h incorrectly in stratified sampling — ensure $\sum n_h = n$.
9. **Reporting only the point estimate** without SE or CI.
10. **Using the wrong CI** — use t when n is small and S^2 is estimated.
11. **Assuming normality for small n** — use bootstrap or exact methods.
12. **Ignoring non-response bias.**
13. **Using SRS formulas for cluster sampling** — `deff` must be applied.
14. **Forgetting that HT is unbiased but can have high variance.**
15. **Mixing up design weights** ($1/\pi_i$) and **size weights** ($1/p_i$).
16. **Using PPS with replacement but applying without-replacement formulas.**
17. **Not checking that** $n_h \geq 2$ for variance estimation.
18. **Using the wrong SE for the total** — remember to multiply by N^2 .
19. **Forgetting to check that** $\sum n_h$ equals the target n .
20. **Interpreting CI coverage as certainty** — it's a long-run property.

23 Part 21 — The Single Most Important Idea

Every sampling question reduces to:

1. **Design:** SRS, stratified, systematic, cluster, two-stage, PPS?
2. **Estimator:** mean, total, proportion, ratio?
3. **Variance:** apply the design-specific formula, include FPC.
4. **CI:** normal or t, with the estimated SE.
5. **Compare:** bias, MSE, relative efficiency, design effect.
6. **Simulate** when theory is hard or you want to validate.

If you can compute a design-correct estimator, its variance, and a CI, you can answer any sampling question.

End of cheat sheet. Good luck in the competition. ““

23.1 What is covered

Section	Topics
Part 0	30-second strategy
Part 1	Foundations: parameters, estimators, FPC, bias/MSE
Part 2	Building finite populations (continuous, stratified, discrete)
Part 3	SRS: sample, mean, variance with FPC, CI, total, proportion, sample size
Part 4	Stratified: proportional, Neyman, optimal with costs, all formulas
Part 5	Systematic: sampling interval, variance approximations
Part 6	Cluster sampling: estimators, variance, deff, intraclass correlation
Part 7	Two-stage sampling: PSU/SSU, variance decomposition
Part 8	Ratio and regression estimators
Part 9	Bias, variance, MSE simulations, relative efficiency
Part 10	Coverage probability simulations
Part 11	Sample size determination (all designs)
Part 12	Design effects and effective sample size
Part 13	Horvitz-Thompson estimator
Part 14	Probability proportional to size (PPS)
Part 15	Non-response and weighting
Part 16	Bootstrap variance and CI
Part 17	Theory Q&A (25 questions)
Part 18	Exam scenarios (10 step-by-step answers)
Part 19	Quick reference table
Part 20	Common mistakes (20 traps)
Part 21	Single most important idea

Design consistency: Every chunk is self-contained with section-prefixed names (`p3_pop`, `p6_m`, `p11_n_h`). Copy-paste any chunk into a fresh R session and it works.